
UGVC-10: Comprehensive System Architecture & Engineering Manual

A Step-by-Step Guide to the Rover's Hardware, Firmware, and Software

Yusuf Borham

Chief Technology Officer (CTO)

AUR Technical Team

May 2026

Abstract

This document serves as the master engineering manual for the UGVC-10 Unmanned Ground Vehicle. It explains the entire system from the flow of electricity on the PCBs, to the embedded C code on the STM32, up to the high-level Artificial Intelligence and ROS 2 Navigation stack on the Raspberry Pi. Sub-team members must read their respective sections to understand how their work integrates into the larger system.

Contents

1	Master System Overview	3
1.1	The Three Layers of the Rover	3
2	Hardware Architecture (Layer 0)	4
2.1	Power Distribution Board (PDB)	4
2.2	The Safety E-Stop Circuit (Crucial)	4
2.3	Logic Level Shifting	4
2.4	Control Board (STM32)	4
2.5	Home Made Motor Driver Boards or BTS	5
3	Firmware Architecture (STM32)	6
3.1	FreeRTOS Task Map	6
3.2	Embedded APIs and Drivers	6
3.2.1	Encoder Driver (Hardware Timers)	6
3.2.2	Motor Control (PWM)	6
3.2.3	Kinematic equation for the rover	6
3.2.4	IMU Driver (BNO055 via I2C)	7
3.2.5	Laser Pointer & Servo Drivers	7
3.2.6	GPS API	7
3.2.7	Communication Module For the PI	7
3.2.8	PID Control for wheel velocity	7
3.2.9	Bluetooth Module for Remote Control	8
3.2.10	Voltage and Current measurement code	8
4	Software Architecture (ROS 2 Humble/Jazzy)	9
4.1	The ROS 2 Workspace (ugvc_ws)	9
4.2	ROS 2 Node & Topic Dictionary	9
4.3	GUI	10
4.4	Network Architecture	10
5	The Autonomy Engine: How We Win	12
5.1	Step 1: Localization (Where am I?)	12
5.2	Step 2: Perception (What is around me?)	12
5.3	Step 3: Planning (How do I move?)	12
5.4	Step 4: The State Machine (Behavior Trees)	12

1 Master System Overview

Building a complex robotic system requires separating the "Brain" from the "Muscles." If the Brain (Raspberry Pi) tries to do muscle work (sending high-speed PWM to motors), it will fail. If the Muscles (STM32) try to do brain work (Computer Vision), they will crash.

1.1 The Three Layers of the Rover

1. **Layer 0 (Hardware):** The skeleton, cardiovascular system, and muscles. Handles raw power, electrical isolation, and physical movement.
2. **Layer 1 (Firmware/STM32):** The spinal cord. Handles real-time reflexes, reads raw sensors instantly, and keeps the motors spinning at exact speeds.
3. **Layer 2 (Software/ROS 2):** The brain. Looks at cameras, builds 3D maps, makes decisions, and tells the spinal cord where to go.

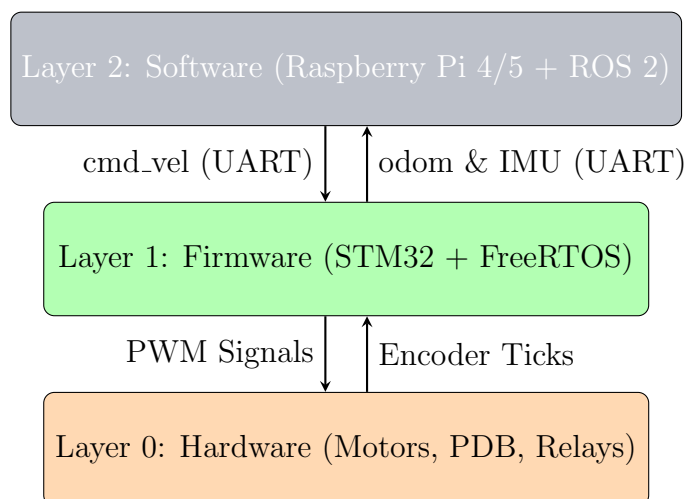


Figure 1: High-Level Architecture and Data Flow

2 Hardware Architecture (Layer 0)

The Hardware team is responsible for ensuring the system does not catch fire, does not suffer from sensor noise, and complies with UGVC safety rules.

2.1 Power Distribution Board (PDB)

The main battery is 24V. We must drop this voltage safely.

- **24V Rail:** Goes directly to the custom Motor Drivers. This is a high-noise environment.
- **5V Rail (Buck Converter):** Powers the Raspberry Pi and the Lidar. Must provide at least 5 Amps to prevent the Pi from rebooting under heavy CPU load.
- **3.3V Rail (Buck Converter):** Powers the STM32, GPS module, and IMU (BNO055). This must be ultra-clean power to prevent sensor drift.

2.2 The Safety E-Stop Circuit (Crucial)

The UGVC rules require two E-Stops (Mechanical and Wireless) that are **hardware-based**.

- **How it works:** The 24V battery ground wire runs through a heavy-duty relay/contacter before reaching the motor drivers.
- **Mechanical Switch:** A big red button on the back of the rover. Hitting it opens the relay.
- **Wireless Switch:** A 2.4GHz LoRa/NRF receiver sits on the rover. If the judge presses the remote, the receiver pulls a pin LOW, which opens the relay.
- *Note:* The Raspberry Pi and STM32 remain powered on when the E-stop is hit. Only the motors lose power. This prevents us from having to reboot the whole computer during a safety pause.

2.3 Logic Level Shifting

The STM32 operates at 3.3V. The Motor Drivers likely require 5V PWM signals. If you connect them directly, the signals will fail or fry the board. Hardware must place **Bi-Directional Logic Level Converters** between the STM32 and the Motor Drivers.

2.4 Control Board (STM32)

The Control Board is based on the STM32 microcontroller, which serves as the central processing unit for the firmware layer.

- **Microcontroller:** STM32F4 series (e.g., STM32F407) for its balance of performance and power efficiency.
- **Communication:** UART for communication with the Raspberry Pi, I2C for sensors, and PWM outputs for motor control.

- **Sensors:** Encoders for wheel speed, IMU (BNO055) for orientation, and GPS module for global positioning.

2.5 Home Made Motor Driver Boards or BTS

- **Motor Drivers:** We use either custom-built H-Bridge motor drivers or off-the-shelf BTS7960 modules, which can handle the high current required by the motors.
- **Features:** These drivers support PWM control for speed regulation and direction control, as well as built-in protection against overcurrent and overheating.
- Also we might need to measure current for better control and safety, so we can add current sensors (like ACS712) to the motor driver circuit.

3 Firmware Architecture (STM32)

The STM32 runs on **FreeRTOS** (Real-Time Operating System). Instead of one giant loop, FreeRTOS splits the code into "Tasks" that run concurrently based on priority.

3.1 FreeRTOS Task Map

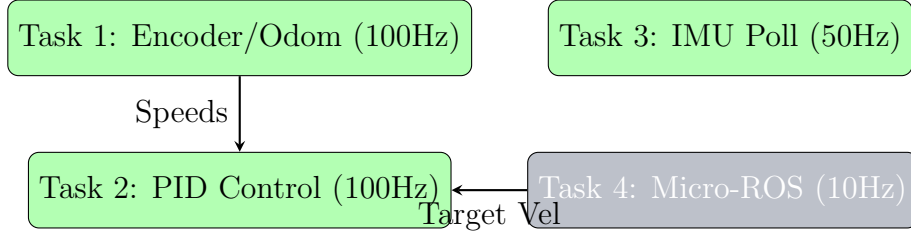


Figure 2: STM32 RTOS Task Priorities

3.2 Embedded APIs and Drivers

Here is exactly how the STM32 interacts with the hardware components:

3.2.1 Encoder Driver (Hardware Timers)

We do not use software interrupts for encoders; it is too slow. We use STM32 Hardware Timers in Encoder Mode.

- **API:** `HAL_TIM_Encoder_Start(&htim1, TIM_CHANNEL_ALL);`
- **Logic:** Read the timer counter (`_HAL_TIM_GET_COUNTER`) every 10ms. Subtract the old value from the new value. Convert those raw ticks into RPM, and then into linear wheel velocity (m/s) using the wheel radius.

3.2.2 Motor Control (PWM)

The STM32 receives a target V_x (forward speed) and ω_z (turning speed) from ROS 2.

- **Kinematics:** We calculate what each wheel should do.

$$V_{left} = V_x - (\omega_z \times \frac{Track_Width}{2})$$
$$V_{right} = V_x + (\omega_z \times \frac{Track_Width}{2})$$

- **API:** `_HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_value);`

3.2.3 Kinematic equation for the rover

The rover is a differential drive robot. The kinematic equations are:

$$V_x = \frac{V_{left} + V_{right}}{2}$$
$$\omega_z = \frac{V_{right} - V_{left}}{Track_Width}$$

These speeds are sent to the PI for the odom message and for the EKF.

3.2.4 IMU Driver (BNO055 via I2C)

The BNO055 does sensor fusion internally.

- **API:** `HAL_I2C_Mem_Read(&hi2c1, BNO_ADDR, REG, 1, buffer, size, timeout);`
- **Logic:** We read the Quaternion registers to get the absolute Heading (Yaw), and the Gyroscope registers to get rotational velocity (ω_z).

3.2.5 Laser Pointer & Servo Drivers

- **Laser:** Just a standard GPIO pin. `HAL_GPIO_WritePin(PORT, PIN, GPIO_PIN_SET);` triggered when ROS 2 says "Face Found."
- **Servo:** Uses a Timer configured for PWM. Modifying the duty cycle changes the angle of the upward-facing camera.

3.2.6 GPS API

- **Module:** We use a u-blox NEO-M8N GPS with RTK capability.
- **Data:** The GPS outputs NMEA sentences over UART. We parse these to extract Latitude, Longitude, and Altitude.

3.2.7 Communication Module For the PI

- **Module:** We use a STM32-based communication module to interface with the Raspberry Pi.
- **Data:** The module handles the wired communication between the STM32 and the Raspberry Pi.

3.2.8 PID Control for wheel velocity

- **Logic:** The PID controller runs at 100Hz. It takes the target wheel velocities from ROS 2 and compares them to the actual velocities from the encoders. The error is processed through the PID algorithm to generate the appropriate PWM signals for the motors.
- It must be tuned carefully to ensure smooth acceleration and deceleration without overshooting or oscillations.
- **API:** The PID output is sent to the motor drivers using the STM32's PWM capabilities, ensuring precise control over the motor speeds.
- Must also include anti windup mechanisms to prevent the integral term from accumulating excessively when the motors are saturated.
- also must have a high pass filter on the derivative term to prevent noise from causing erratic motor behavior.

3.2.9 Bluetooth Module for Remote Control

- **Module:** We use a Bluetooth module (e.g., HC-05) for remote control and diagnostics.
- **Data:** The module allows for wireless communication between the STM32 and a smartphone or laptop, enabling manual control and real-time monitoring of sensor data, or for taking the robot from one place to another without having to start the whole system up againn

3.2.10 Voltage and Current measurement code

- Probably we will use a voltage divider with a protection circuit with an ADC pin on the STM32 to measure the battery voltage. For current measurement, we can use a current sensor like the ACS712, which outputs an analog voltage proportional to the current flowing through it. The STM32 can read this voltage using its ADC to calculate the current draw of the motors and other components, or we can add a shunt resistor in series with the motor power line and measure the voltage drop across it to calculate current using Ohm's Law ($I = V/R$).

4 Software Architecture (ROS 2 Humble/Jazzy)

The Raspberry Pi handles all the complex AI, mapping, and decision-making.

4.1 The ROS 2 Workspace (`ugvc_ws`)

The workspace must be strictly organized into these packages:

- **`ugvc_description`:** Contains the URDF file. This is an XML file that tells ROS 2 exactly where every sensor is physically bolted on the robot (e.g., "The GPS is 20cm behind the center of the wheels").
- **`ugvc_hardware`:** Runs the `micro_ros_agent` to talk to the STM32.
- **`ugvc_vision`:** Contains the Python OpenCV and YOLOv8 scripts.
- **`ugvc_navigation`:** Contains the Kalman Filters, Costmaps, and Path Planners.

4.2 ROS 2 Node & Topic Dictionary

This is the lifeblood of the system. Every team member must know what data is flowing where.

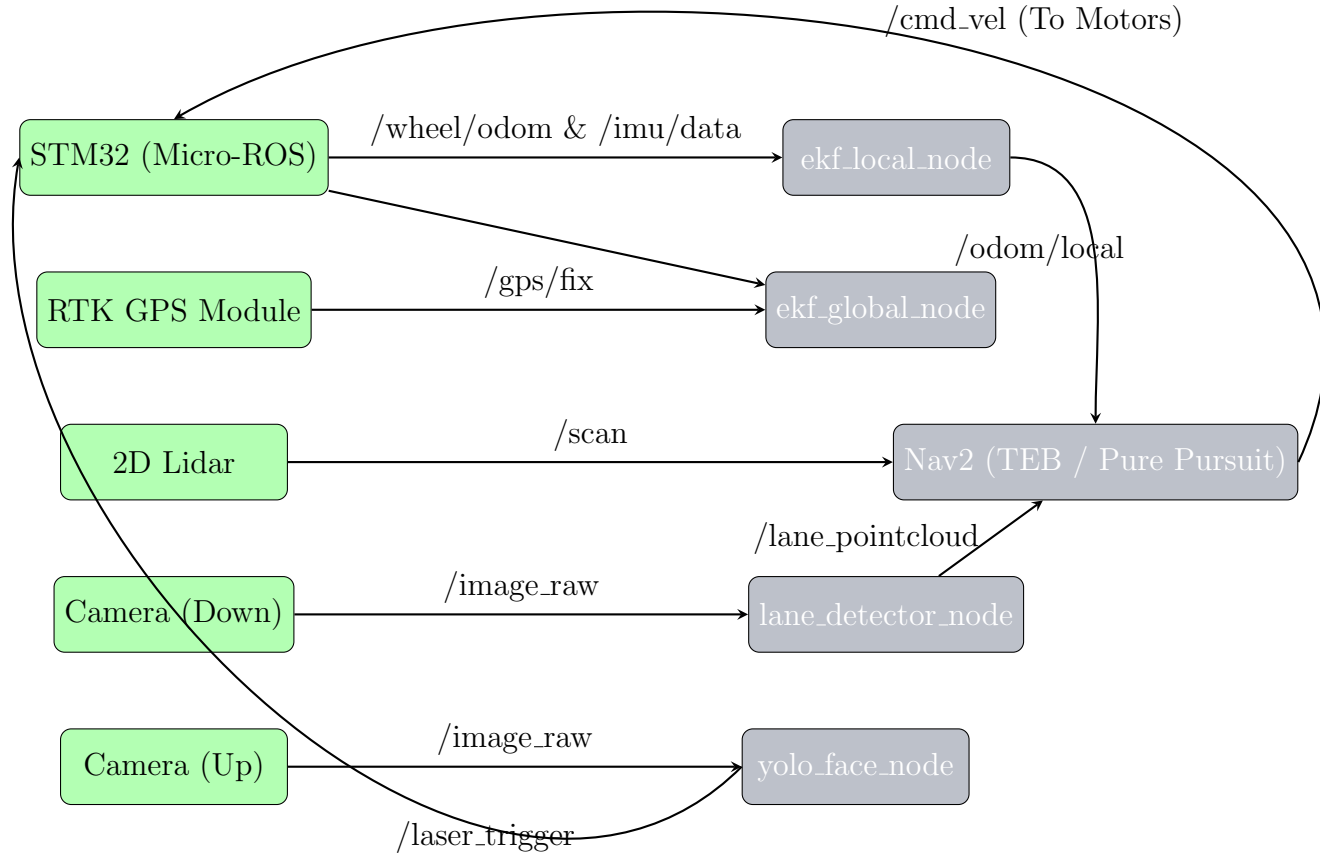


Figure 3: Complete ROS 2 Node and Topic Architecture

4.3 GUI

We have a custom GUI built with **rqt** that shows:

- Real-time camera feeds with lane detection and face bounding boxes.
- A live-updating map with the rover's position, Lidar obstacles, and planned path.
- Diagnostic data like wheel speeds, GPS coordinates, and IMU readings.

4.4 Network Architecture

The Raspberry Pi runs a ROS 2 **DDS** (Data Distribution Service) network. All nodes communicate over this network using topics. The micro-ROS agent or the pyserial on the STM32 bridges the hardware data into this network.

A long Range router to allow the station to be connected to the rover from a distance for the camera feed and the manual control if needed.

5 The Autonomy Engine: How We Win

5.1 Step 1: Localization (Where am I?)

We use the **Extended Kalman Filter (EKF)** from the `robot_localization` package.

- **Local EKF:** Fuses the Encoders + IMU. It provides a super smooth 50Hz location update so the rover doesn't jerk around obstacles.
- **Global EKF:** Fuses Encoders + IMU + RTK GPS. This locks the rover to the exact Earth coordinates to hit the 1.5m waypoint target.

5.2 Step 2: Perception (What is around me?)

The UGVC course has physical objects, flat painted white lines, and flat white potholes.

- **The Lidar:** Maps the 3D physical obstacles into the ROS 2 **Costmap**.
- **The Downward Camera:** Runs an OpenCV Python script. It masks the image for pure white. It uses `cv2.warpPerspective` to flatten the image (Bird's Eye View). It then publishes the white pixels as a fake 3D Pointcloud.
- **The Result:** The ROS 2 Costmap merges the Lidar and the Camera data. To the path planner, a flat white painted line looks exactly like a 10-foot brick wall. It will not cross it.

5.3 Step 3: Planning (How do I move?)

- **Global Planner (A* or Dijkstra):** Draws the shortest line from where we are to the GPS waypoint.
- **Local Planner (TEB Local Planner):** Looks at the Costmap and the Global Path. It bends the path around the obstacles and white lines dynamically. It guarantees we stay between 2 km/hr and 10 km/hr and don't accelerate too fast.

5.4 Step 4: The State Machine (Behavior Trees)

We do not use messy "if/else" statements in Python. We use Nav2 **Behavior Trees (XML)**.

1. **Bounded Track Mode:** Use Lidar + Camera + TEB Planner to dodge obstacles.
2. **Unbounded Waypoint Mode:** Turn off Cameras. Turn off TEB. Use **Pure Pursuit** to drive full speed directly to the GPS coordinates (because this area is obstacle-free).
3. **Waypoint 2 (Image Detection):**
 - Halt `cmd_vel`.
 - Trigger STM32 to sweep the servo.
 - `yolo_face_node` looks for the target dummy.

- When found, publish `True` to `/laser_trigger`.
- Wait 3 seconds, resume driving to Waypoint 3.